
作业报告

课程名称： 算法设计与分析

作业题目： 数字三角形

专业：

班级：

学号：

姓名：

联系方式：

邮箱：

2025 年 12 月 2 日

1.概述思路

这是一个动态规划问题，进行最优子结构分析。

最优子结构是动态规划问题的核心特性。一个问题如果能够通过子问题的最优解来构建整体问题的最优解，则这个问题满足最优子结构性质。

定义：设 $x[i][j]$ 表示三角形第 i 行第 j 个位置的数字。我们需要计算从三角形顶端到底端的路径和的最大值。

子问题：目标是从顶部的数字 $x[1][1]$ 到达底部的数字 $x[n][k]$ ，可以通过分解这个问题来得到子问题的最优解。

具体来说：如果我们在某一位置 $x[i][j]$ ，那么最大路径和可以通过从下一行的两个位置获得：位置 $x[i+1][j]$ （直接向下）位置 $x[i+1][j+1]$ （右下方）。

因此，当前位置的最优解（即最大路径和）可以通过选择下一行中这两个位置的最大值加上当前值得到。

采用备忘录 $s[i][j]$ 来记录选择的路径是下一行的哪个。递推公式如下：

$$m[i][j] = x[i][j] + \max(x[i+1][j], x[i+1][j+1])$$

$$s[i][j] = m[i+1][j] > m[i+1][j+1] ? j : j+1$$

这里 $m[i][j]$ 表示从 $x[i][j]$ 到达底部的最大路径和。

总结：当前节点的最优解（最大路径和）是由下一行的两个子问题解的最大值加上当前值得到的。这种结构是**最优子结构**，因为我们可以通过子问题的最优解来得到当前问题的最优解。所以我们可以使用动态规划来解决。

2.代码

```
#include <iostream>
#include <string>
#include <math.h>
#include <numeric>
using namespace std;

int n,m[100][100],s[100][100],x[100][100];

int main() {
    /*
        7
       3 8
      8 1 0
     2 7 4 4
    4 5 2 6 5
    */
    cin >> n;
    for(int i = 1; i <= n; i++){
        for (int j = 1; j <= i; j++)
            cin >> x[i][j];
```

```

}

//初始化最底层
for(int j = 1; j <= n; j++) m[n][j] = x[n][j];

for(int i = n-1 ; i > 0; i--){
    for(int j = 1; j <= i; j++){
        m[i][j] = x[i][j] + max(m[i+1][j],m[i+1][j+1]); //计算最大值
        s[i][j] = m[i+1][j] > m[i+1][j+1] ? j : j+1; //记录路径
    }
}

cout << m[1][1] << endl;
cout << "路径: " << endl;
for(int i = 1, j = 1; i <= n; i++){
    cout << x[i][j] << " ";
    j = s[i][j];
}
}

```

复杂度分析

时间复杂度： $O(n^2)$ ：代码中用于动态规划的两层循环：for (int i = n-1; i > 0; i--), for (int j = 1; j <= i; j++)会遍历整个三角形中除最后一行外的所有位置，数量约为 $\frac{n(n-1)}{2}$ ，因此时间复杂度为 $O(n^2)$ 。另外最后输出路径的循环是 $O(n)$ ，不影响总体。

空间复杂度： $O(n^2)$ ：使用了三个二维数组 x、m、s 存储三角形数值、DP 值与路径记录，规模都是 100×100 （一般按 $n \times n$ 计），因此空间复杂度为 $O(n^2)$ 。

3.测试结果

样例测试：

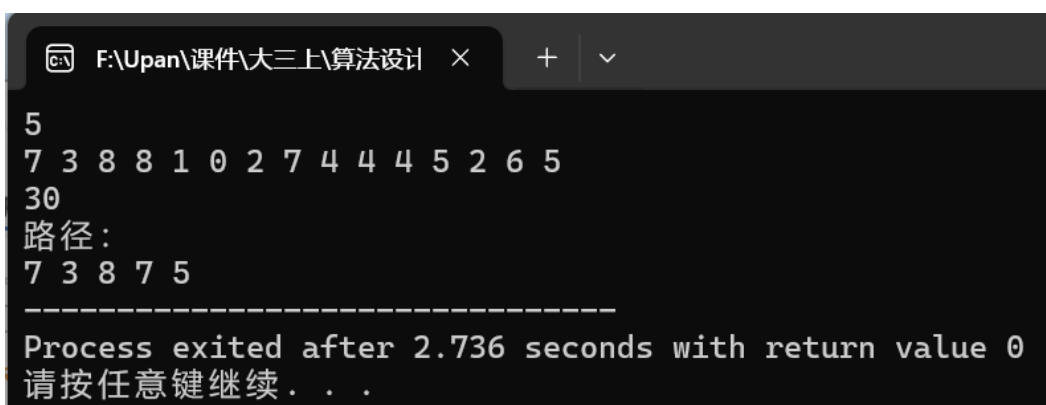
输入如下：

```

5
7 3 8 8 1 0 2 7 4 4 4 5 2 6 5

```

运行结果：

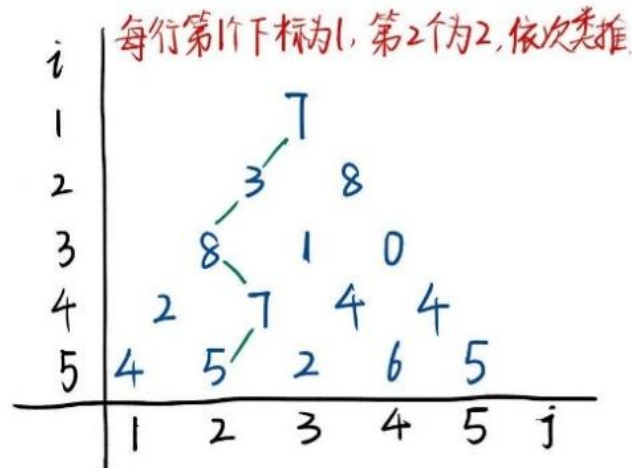


```

F:\Upan\课件\大三上\算法设计 x + v
5
7 3 8 8 1 0 2 7 4 4 4 5 2 6 5
30
路径:
7 3 8 7 5
-----
Process exited after 2.736 seconds with return value 0
请按任意键继续. . .

```

顺序如下图所示：



对应的 m 和 s 数组如下：

m	1	2	3	4	5
1	30				
2	23	21			
3	20	13	10		
4	7	12	10	10	
5	4	5	2	6	5

s	1	2	3	4	5
1	[2,1]				
2	[3,1]	[3,1]			
3	[4,2]	[4,1]	[4,3]		
4	[5,2]	[5,2]	[5,4]	[5,4]	
5		x			

$$m[i][j] = x[i][j] + \max(x[i+1][j], x[i+1][j+1])$$

$$s[i][j] = m[i+1][j] > m[i+1][j+1] ? j : j+1$$

测试用例 1：

4
1
2 2
3 1 3
4 4 4 4

结果如图：

```

F:\Upan\课件\大三上\算法设计  ×  +  v
4
1
2 2
3 1 3
4 4 4 4
10
路径:
1 2 3 4
-----
Process exited after 2.61 seconds with return value 0
请按任意键继续. . .

```

测试用例 2:

```

6
5
9 6
4 6 8
0 7 1 5
4 5 2 6 5
6 6 6 6 6 6

```

结果如图:

```

F:\Upan\课件\大三上\算法设计  ×  +  v
6
5
9 6
4 6 8
0 7 1 5
4 5 2 6 5
6 6 6 6 6 6
38
路径:
5 9 6 7 5 6
-----
Process exited after 4.062 seconds with return value 0
请按任意键继续. . .

```

测试用例 3:

```

7
6
3 8
9 1 5
4 7 2 6
8 5 9 3 7
2 6 4 8 1 5
7 3 2 9 6 4 8

```

结果如图：

```

F:\Upan\课件\大三上\算法设计  ×  +  v
7
6
3 8
9 1 5
4 7 2 6
8 5 9 3 7
2 6 4 8 1 5
7 3 2 9 6 4 8
51
路径：
6 3 9 7 9 8 9
-----
Process exited after 2.151 seconds with return value 0
请按任意键继续. . .

```

测试用例 4：

```

10
75
95 64
17 47 82
18 35 87 10
20 04 82 47 65
19 01 23 75 03 34
88 02 77 73 07 63 67
99 65 04 28 06 16 70 92
41 41 26 56 83 40 80 70 33
41 48 72 33 47 32 37 16 94 29

```

结果如图：

```

F:\Upan\课件\大三上\算法设计  ×  +  v
10
75
95 64
17 47 82
18 35 87 10
20 04 82 47 65
19 01 23 75 03 34
88 02 77 73 07 63 67
99 65 04 28 06 16 70 92
41 41 26 56 83 40 80 70 33
41 48 72 33 47 32 37 16 94 29
696
路径：
75 64 82 87 82 75 73 28 83 47
-----
Process exited after 1.444 seconds with return value 0
请按任意键继续. . .

```